

Computer Vision H.W. #3

Student ID and Names: -

b00094286 --- Mohamed Abdelrahim Alawadhi

b00097405 --- Arya Sankhe

✓ Question 1: Building a Dense Neural Network (ANN) for Fashion Classification

Dataset: [Fashion-MNIST](#)

This dataset consists of 60,000 training and 10,000 testing grayscale images, across 10 categories of clothing items (e.g., T-shirts, trousers, bags). Task: Create a dense neural network to classify images into one of the 10 fashion categories. Design a network with:

- An input layer with the correct size .
- At least two hidden layers with ReLU activation.
- An output layer with an activation for multi-class classification.
- Build 2 models with 2 different optimizers SGD and adam and **comment on the results**.
- Report metrics: accuracy, loss, precision, recall, and F1-score.
- For accuracy include training and testing accuracy graphs.

```
import torch
from torch import nn
from torch import optim
from torch.utils.data import Dataset
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision import transforms
from torchvision.transforms import ToTensor
from sklearn.metrics import precision_score, recall_score, f1_score

import numpy as np
from matplotlib import pyplot as plt

# Setting a random seed for reproducibility
torch.manual_seed(0)

# Defining the data transformation.
transform = transforms.Compose([transforms.ToTensor(),
                                transforms.Normalize(
                                    mean = (0.5,),
                                    std = (0.5,))
                                ])

# Loading in the FashionMNIST dataset.
trainset = datasets.FashionMNIST(
    root = "./data",
    train = True,
    download = True,
    transform = transform
)

testset = datasets.FashionMNIST(
    root = "./data",
    train = False,
    download = True,
    transform = transform
)

# Setting the batch size, along with creating the dataloader objects for both training and testing.
batchsize = 64
trainloader = DataLoader(trainset, batch_size = batchsize, shuffle = True)
testloader = DataLoader(testset, batch_size = batchsize, shuffle = False)

# Checking if the MPS is available. If not, it will fallback to the CPU.
device = torch.device("mps" if torch.backends.mps.is_available() else "cuda" if torch.cuda.is_available() else "cpu")
```

```

# Utilized Chatgpt to create suitable and effective functions.
# Creating a function that defines the model.
def createModel():
    model = nn.Sequential(
        nn.Flatten(),
        nn.Linear(28 * 28, 128),
        nn.ReLU(),
        nn.Linear(128, 64),
        nn.ReLU(),
        nn.Linear(64, 10),
        nn.Softmax(dim = 1)
    )
    return model.to(device)

# Creating a function that calculates the accuracy, precision, recall, and F1-score
def calculateMetrics(y_true, y_pred):
    accuracy = (y_true == y_pred).sum().item() / len(y_true)
    precision = precision_score(y_true, y_pred, average = "weighted", zero_division = 1)
    recall = recall_score(y_true, y_pred, average = "weighted", zero_division = 1)
    f1 = f1_score(y_true, y_pred, average = "weighted", zero_division = 1)
    return accuracy, precision, recall, f1

# Creating a function that trains and evaluates.
def trainAndEvaluate(model, optimizer, numEpochs = 10):
    model.train()
    train_history = []
    val_history = []
    train_accuracies = []
    val_accuracies = []

    for epoch in range(numEpochs):
        # Training loop.
        train_loss = 0.0
        correct_train = 0
        total_train = 0
        y_true_train = []
        y_pred_train = []

        for i, data in enumerate(trainloader):
            inputs, labels = data
            inputs, labels = inputs.to(device), labels.to(device)

            optimizer.zero_grad()
            outputs = model(inputs)

            loss = criteria(outputs, labels)
            loss.backward()

            optimizer.step()

            train_loss += loss.item()

            _, predicted = torch.max(outputs, 1)
            correct_train += (predicted == labels).sum().item()
            total_train += labels.size(0)

            y_true_train.extend(labels.cpu().numpy())
            y_pred_train.extend(predicted.cpu().numpy())

        train_accuracy, train_precision, train_recall, train_f1 = calculateMetrics(np.array(y_true_train), np.array(y_pred_train))
        train_history.append(train_loss / len(trainloader))
        train_accuracies.append(train_accuracy)

    # Validation loop.
    model.eval()
    val_loss = 0.0
    correct_val = 0
    total_val = 0
    y_true_val = []
    y_pred_val = []

    with torch.no_grad():
        for data in testloader:
            inputs, labels = data
            inputs, labels = inputs.to(device), labels.to(device)

            outputs = model(inputs)
            loss = criteria(outputs, labels)

```

```
val_loss += loss.item()

_, predicted = torch.max(outputs, 1)
correct_val += (predicted == labels).sum().item()
total_val += labels.size(0)

y_true_val.extend(labels.cpu().numpy())
y_pred_val.extend(predicted.cpu().numpy())

val_accuracy, val_precision, val_recall, val_f1 = calculateMetrics(np.array(y_true_val), np.array(y_pred_val))
val_history.append(val_loss / len(testloader))
val_accuaries.append(val_accuracy)

# Printing out the metrics for the epoch.
print(f"Epoch [{epoch+1}/{numEpochs}], "
      f"Train Loss: {train_loss / len(trainloader):.4f}, Train Accuracy: {train_accuracy:.4f}, "
      f"Validation Loss: {val_loss / len(testloader):.4f}, Validation Accuracy: {val_accuracy:.4f}, "
      f"Precision: {val_precision:.4f}, Recall: {val_recall:.4f}, F1: {val_f1:.4f}")

return train_accuaries, val_accuaries, train_history, val_history

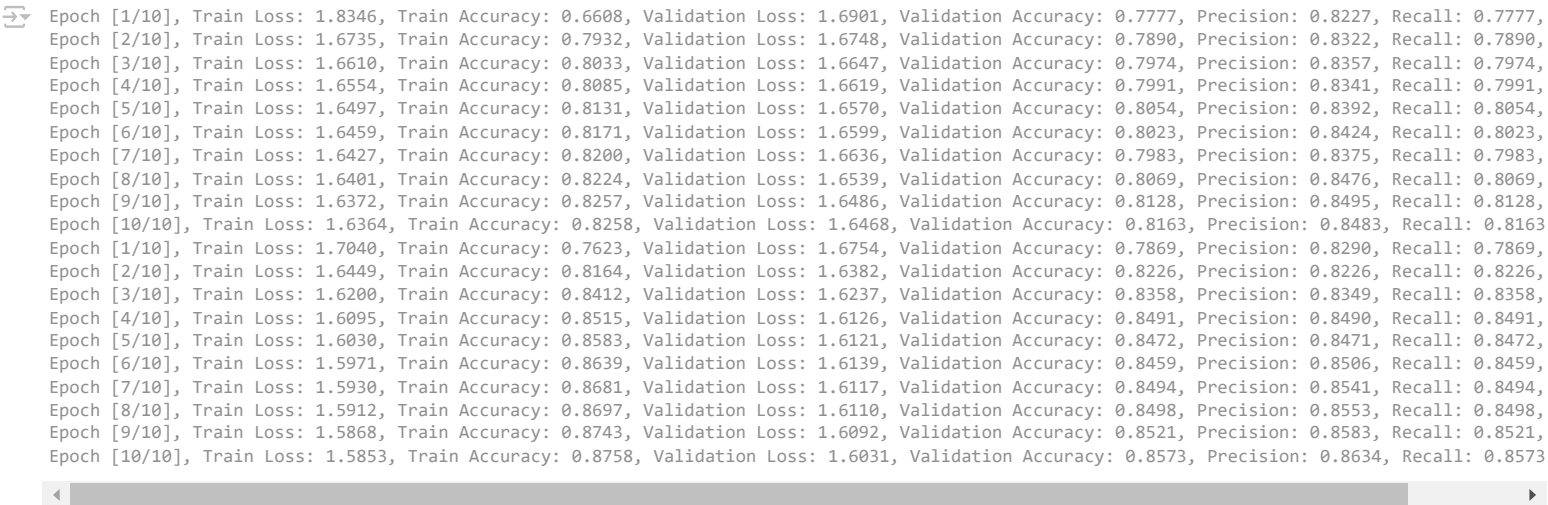
criteria = nn.CrossEntropyLoss()

# Initializing the model and train with both optimizers.
model = createModel()

# Optimizer 1: SGD
optimizer_sgd = optim.SGD(model.parameters(), lr = 0.01, momentum = 0.9)
train_sgd_acc, val_sgd_acc, train_sgd_loss, val_sgd_loss = trainAndEvaluate(model, optimizer_sgd, numEpochs = 10)

# Reinitializing the model for the Adam optimizer.
model = createModel()

# Optimizer 2: Adam
optimizer_adam = optim.Adam(model.parameters(), lr = 0.001)
train_adam_acc, val_adam_acc, train_adam_loss, val_adam_loss = trainAndEvaluate(model, optimizer_adam, numEpochs = 10)
```



Epoch	Train Loss	Train Accuracy	Validation Loss	Validation Accuracy	Precision	Recall	F1
[1/10]	1.8346	0.6608	1.6901	0.7777	0.8227	0.7777	0.8000
[2/10]	1.6735	0.7932	1.6748	0.7890	0.8322	0.7890	0.8106
[3/10]	1.6610	0.8033	1.6647	0.7974	0.8357	0.7974	0.8165
[4/10]	1.6554	0.8085	1.6619	0.7991	0.8341	0.7991	0.8166
[5/10]	1.6497	0.8131	1.6570	0.8054	0.8392	0.8054	0.8223
[6/10]	1.6459	0.8171	1.6599	0.8023	0.8424	0.8023	0.8223
[7/10]	1.6427	0.8200	1.6636	0.7983	0.8375	0.7983	0.8179
[8/10]	1.6401	0.8224	1.6539	0.8069	0.8476	0.8069	0.8272
[9/10]	1.6372	0.8257	1.6486	0.8128	0.8495	0.8128	0.8361
[10/10]	1.6364	0.8258	1.6468	0.8163	0.8483	0.8163	0.8323
[1/10]	1.7040	0.7623	1.6754	0.7869	0.8290	0.7869	0.8079
[2/10]	1.6449	0.8164	1.6382	0.8226	0.8226	0.8226	0.8226
[3/10]	1.6200	0.8412	1.6237	0.8358	0.8349	0.8358	0.8353
[4/10]	1.6095	0.8515	1.6126	0.8491	0.8491	0.8491	0.8491
[5/10]	1.6030	0.8583	1.6121	0.8472	0.8472	0.8472	0.8472
[6/10]	1.5971	0.8639	1.6139	0.8459	0.8506	0.8459	0.8484
[7/10]	1.5930	0.8681	1.6117	0.8494	0.8541	0.8494	0.8518
[8/10]	1.5912	0.8697	1.6110	0.8498	0.8553	0.8498	0.8525
[9/10]	1.5868	0.8743	1.6092	0.8521	0.8583	0.8521	0.8552
[10/10]	1.5853	0.8758	1.6031	0.8573	0.8634	0.8573	0.8603

Report Matrics

- It seems that the Adam optimizer has much higher training and validation accuracy as compared to the SGD.
- The Adam optimizer had lower loss in both the training and validation losses.
- As well as, the Adam optimizer has higher precision, recall, and F1-score that reveals a better performance overall on the FashionMNIST dataset.

Conclusion: -

The Adam optimizer has a better overall performance with faster convergence and higher accuracy than the SGD optimizer. Therefore, it would make sense, that the Adam optimizer would be the perferred optimizer for this dataset

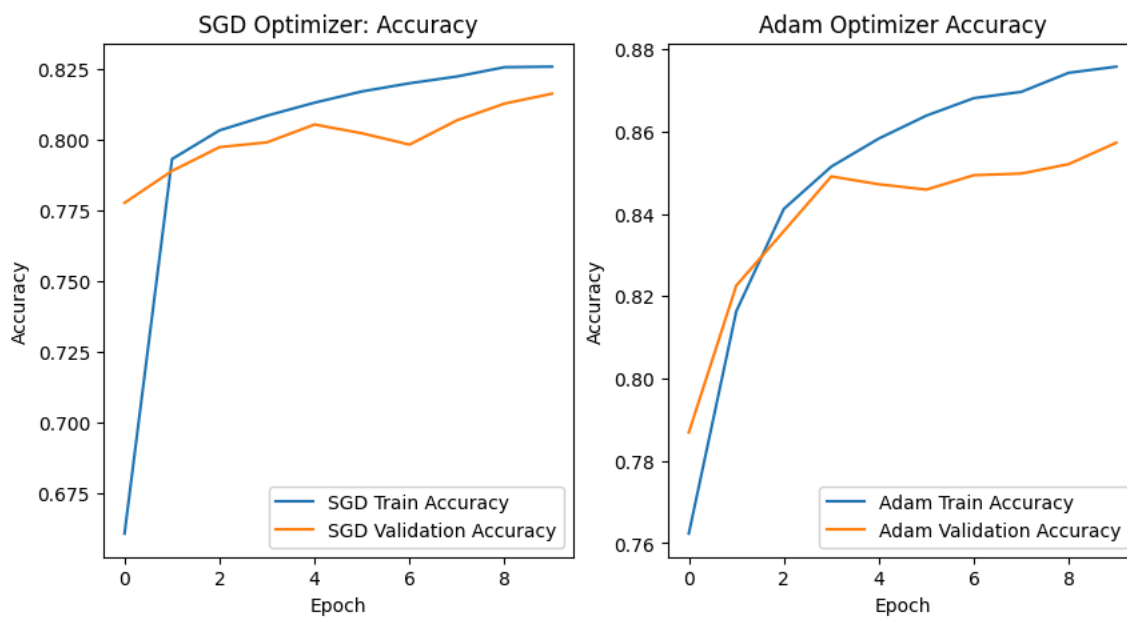
Epoch	Optimizer	Train Accuracy (%)	Validation Accuracy (%)	Train Loss	Validation Loss	Precision (%)	Recall (%)	F1-Score (%)
1	SGD	66.08	77.77	1.8346	1.6901	82.27	77.77	74.35
10	SGD	82.58	81.63	1.6364	1.6468	84.83	81.63	77.81
1	Adam	76.23	78.69	1.7040	1.6754	82.90	78.69	75.08
10	Adam	87.58	85.73	1.5853	1.6031	86.34	85.73	85.82

```
# Plotting the accuracy of both optimizers.
plt.figure(figsize = (10, 5))
```

```
# SGD plot.
plt.subplot(1,2,1)
plt.plot(train_sgd_acc, label = "SGD Train Accuracy")
plt.plot(val_sgd_acc, label = "SGD Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.title("SGD Optimizer: Accuracy")
```

```
# Adam plot.
plt.subplot(1,2,2)
plt.plot(train_adam_acc, label = "Adam Train Accuracy")
plt.plot(val_adam_acc, label = "Adam Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.title("Adam Optimizer Accuracy")
```

```
plt.show()
```



Question 2: Designing a Convolutional Neural Network for CIFAR-10 Image Classification

Dataset: [CIFAR-10](#)

This dataset contains 60,000 images in 10 classes (e.g., airplanes, birds, cats).

Create a CNN to classify images into the 10 CIFAR-10 classes.

The CNN should include:

- Input and at least two convolutional layers with ReLU activation and pooling.
- Fully connected layers for the final classification.
- Train the model, then report metrics: accuracy, loss, precision, recall, and F1-score.
- For accuracy include training and testing accuracy graphs.

```
torch.manual_seed(0)
```

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean = [0.5, 0.5, 0.5], std = [0.5, 0.5, 0.5])
])
```

```
trainset = datasets.CIFAR10(
    root = "./data2",
    train = True,
    download = True,
    transform = transform
)
```

```
testset = datasets.CIFAR10(
    root = "./data2",
    train = False,
    download = True,
    transform = transform
)
```

```
batch_size = 64
trainloader = DataLoader(trainset, batch_size = batch_size, shuffle = True)
testloader = DataLoader(testset, batch_size = batch_size, shuffle = False)
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```



Files already downloaded and verified
Files already downloaded and verified

```
class MyCnnModel(nn.Module):
    def __init__(self):
        super(MyCnnModel, self).__init__()

        # Layer 1
        self.conv1 = nn.Conv2d(in_channels = 3, out_channels = 32, kernel_size = 3, padding = 1)
        self.relu1 = nn.ReLU()
        self.pool1 = nn.MaxPool2d(2, 2)

        # Layer 2
        self.conv2 = nn.Conv2d(in_channels = 32, out_channels = 64, kernel_size = 3, padding = 1)
        self.relu2 = nn.ReLU()
        self.pool2 = nn.MaxPool2d(2, 2)

        # Layer 3
        self.conv3 = nn.Conv2d(in_channels = 64, out_channels = 128, kernel_size = 3, padding = 1)
        self.relu3 = nn.ReLU()
        self.pool3 = nn.MaxPool2d(2, 2)

        # Connected layers.
        self.fc1 = nn.Linear(128 * 4 * 4, 512)
        self.fc2 = nn.Linear(512, 10)

    def forward(self, x):
        x = self.pool1(self.relu1(self.conv1(x)))
        x = self.pool2(self.relu2(self.conv2(x)))
        x = self.pool3(self.relu3(self.conv3(x)))

        # Flattening the tensor before adding it to the connected layers.
        x = x.view(-1, 128 * 4 * 4)

        # Applying the fully connected layers.
        x = self.fc1(x)
        x = self.fc2(x)

        return x

# Initializing the model.
model = MyCnnModel()
model.to(device)

# The loss function and the optimizer.
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr = 0.001)

train_acc_history = []
test_acc_history = []
```

```

numEpochs = 10
for epoch in range(numEpochs):
    model.train()
    correct_train = 0
    total_train = 0
    train_loss = 0.0

    for i, (inputs, labels) in enumerate(trainloader):
        inputs, labels = inputs.to(device), labels.to(device)

        optimizer.zero_grad()

        outputs = model(inputs)

        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        _, predicted = torch.max(outputs, 1)
        correct_train += (predicted == labels).sum().item()
        total_train += labels.size(0)

        train_loss += loss.item()

train_accuracy = correct_train / total_train
train_acc_history.append(train_accuracy)

model.eval()
correct_test = 0
total_test = 0
with torch.no_grad():
    for inputs, labels in testloader:
        inputs, labels = inputs.to(device), labels.to(device)
        outputs = model(inputs)
        _, predicted = torch.max(outputs, 1)
        correct_test += (predicted == labels).sum().item()
        total_test += labels.size(0)

test_accuracy = correct_test / total_test
test_acc_history.append(test_accuracy)

print(f"Epoch [{epoch+1}/{numEpochs}], "
      f"Train Loss: {train_loss/len(trainloader):.4f}, "
      f"Train Accuracy: {train_accuracy:.4f}, "
      f"Test Accuracy: {test_accuracy:.4f}")

```

Epoch [1/10], Train Loss: 1.2750, Train Accuracy: 0.5401, Test Accuracy: 0.6429
 Epoch [2/10], Train Loss: 0.8784, Train Accuracy: 0.6919, Test Accuracy: 0.6970
 Epoch [3/10], Train Loss: 0.7226, Train Accuracy: 0.7483, Test Accuracy: 0.7296
 Epoch [4/10], Train Loss: 0.6291, Train Accuracy: 0.7819, Test Accuracy: 0.7395
 Epoch [5/10], Train Loss: 0.5520, Train Accuracy: 0.8054, Test Accuracy: 0.7565
 Epoch [6/10], Train Loss: 0.4854, Train Accuracy: 0.8304, Test Accuracy: 0.7375
 Epoch [7/10], Train Loss: 0.4304, Train Accuracy: 0.8482, Test Accuracy: 0.7461
 Epoch [8/10], Train Loss: 0.3811, Train Accuracy: 0.8645, Test Accuracy: 0.7502
 Epoch [9/10], Train Loss: 0.3331, Train Accuracy: 0.8813, Test Accuracy: 0.7406
 Epoch [10/10], Train Loss: 0.2995, Train Accuracy: 0.8923, Test Accuracy: 0.7441

```

# Creating a function that calculates the precision, recall, and F1-score
def calculateMetrics(y_true, y_pred):
    precision = precision_score(y_true, y_pred, average = "weighted", zero_division = 1)
    recall = recall_score(y_true, y_pred, average = "weighted", zero_division = 1)
    f1 = f1_score(y_true, y_pred, average = "weighted", zero_division = 1)
    return precision, recall, f1

```

```

model.eval()
y_true = []
y_pred = []

with torch.no_grad():
    for inputs, labels in testloader:
        inputs, labels = inputs.to(device), labels.to(device)
        outputs = model(inputs)
        _, predicted = torch.max(outputs, 1)
        y_true.extend(labels.cpu().numpy())
        y_pred.extend(predicted.cpu().numpy())

precision, recall, f1 = calculateMetrics(np.array(y_true), np.array(y_pred))

print(f"Precision: {precision:.4f}, Recall: {recall:.4f}, F1-Score: {f1:.4f}")

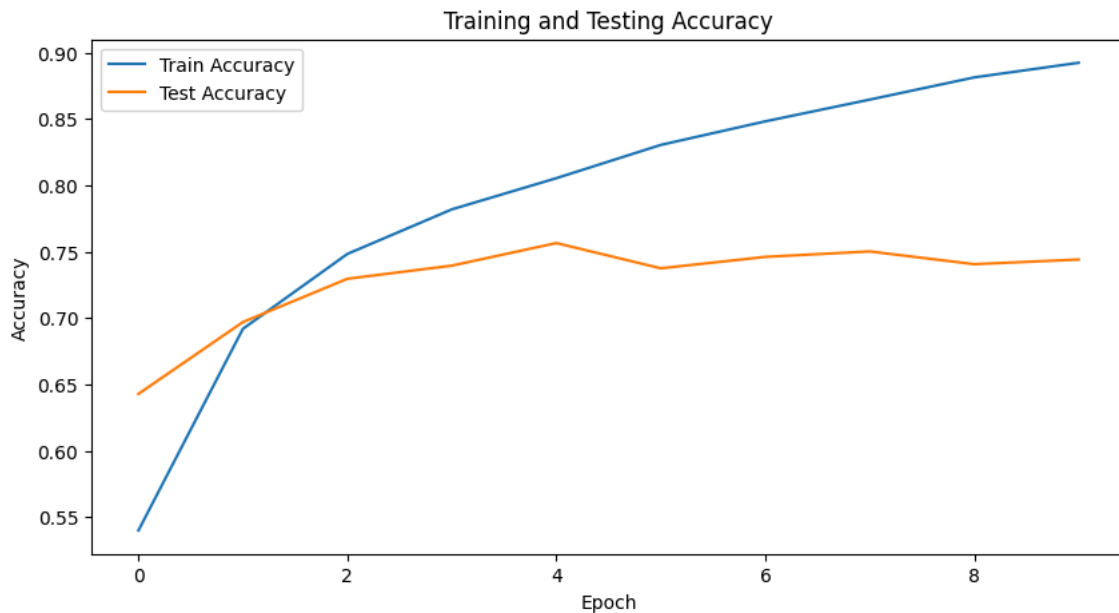
```

Precision: 0.7469, Recall: 0.7441, F1-Score: 0.7447

```
plt.figure(figsize = (10, 5))
```

```
plt.plot(train_acc_history, label = "Train Accuracy")
plt.plot(test_acc_history, label = "Test Accuracy")
```

```
plt.title("Training and Testing Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```



Question 3 Transfer Learning with Data Augmentation and Hyperparameter Tuning

1. Download and Load the Dataset:

Download the Intel Image Classification dataset and organize it into training, validation, and testing splits. Load the images with the `torchvision.datasets.ImageFolder` class.

2. Data Augmentation:

Apply transformations such as random horizontal flips, rotations, and color jitter to increase variability in the training set, using `torchvision.transforms`.

3. Pre-trained Model Selection and Customization:

Load a pre-trained ResNet and replace the final layer with a new fully connected layer to match the 6 classes. Freeze the weights of the initial layers to retain high-level features, and only train the final layers for the first experiment.

4. Hyperparameter Tuning with Ray Tune:

Use Ray Tune to tune the hyperparameters, such as learning rate, batch size, and the number of neurons in the last layers, and evaluate different optimizer choices (e.g., Adam, SGD).

5. Report your results before and after tuning

```
import torch
import torchvision
import torchvision.transforms as transforms
from torchvision import datasets
from torch.utils.data import DataLoader, random_split
import os
import zipfile
import requests

# Step 1: Download the dataset
url = "https://storage.googleapis.com/intel-image-classification/seg_train.zip"
dataset_zip = "seg_train.zip"
```

```

if not os.path.exists(dataset_zip):
    print("Downloading dataset...")
    r = requests.get(url)
    with open(dataset_zip, 'wb') as f:
        f.write(r.content)

# Step 2: Extract the dataset
if not os.path.exists("seg_train"):
    print("Extracting dataset...")
    try:
        with zipfile.ZipFile(dataset_zip, 'r') as zip_ref:
            zip_ref.extractall()
    except zipfile.BadZipFile:
        raise Exception("The downloaded file is not a valid zip file. Please check the URL or download the file again.")

# Step 3: Define transforms (including data augmentation)
data_transforms = transforms.Compose([
    transforms.Resize((224, 224)), # Resize to match common input size for pretrained models
    transforms.RandomHorizontalFlip(), # Data augmentation
    transforms.RandomRotation(10), # Data augmentation
    transforms.ToTensor(), # Convert to tensor
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]) # Normalization for pretrained models
])

# Step 4: Load dataset using ImageFolder
dataset_path = "seg_train/seg_train"
dataset = datasets.ImageFolder(root=dataset_path, transform=data_transforms)

# Step 5: Split dataset into training, validation, and testing sets
data_size = len(dataset)
train_size = int(0.7 * data_size)
val_size = int(0.2 * data_size)
test_size = data_size - train_size - val_size

train_data, val_data, test_data = random_split(dataset, [train_size, val_size, test_size])

# Step 6: Create DataLoaders for each split
batch_size = 32
train_loader = DataLoader(train_data, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_data, batch_size=batch_size, shuffle=False)
test_loader = DataLoader(test_data, batch_size=batch_size, shuffle=False)

print("Data loaded successfully:")
print(f"Training set: {len(train_data)} images")
print(f"Validation set: {len(val_data)} images")
print(f"Testing set: {len(test_data)} images")

```



Extracting dataset...

```

-----
BadZipFile                                Traceback (most recent call last)
<ipython-input-3-a29f838e2df3> in <cell line: 21>()
    23     try:
--> 24         with zipfile.ZipFile(dataset_zip, 'r') as zip_ref:
    25             zip_ref.extractall()

```

2 frames

BadZipFile: File is not a zip file

During handling of the above exception, another exception occurred:

```

Exception                                Traceback (most recent call last)
<ipython-input-3-a29f838e2df3> in <cell line: 21>()
    25         zip_ref.extractall()
    26     except zipfile.BadZipFile:
--> 27         raise Exception("The downloaded file is not a valid zip file. Please check the URL or download the file again.")
    28
    29

```

Exception: The downloaded file is not a valid zip file. Please check the URL or download the file again.

✓ Other Attempt on question 3 using Chatgpt.

```

import os
import shutil
from sklearn.model_selection import train_test_split

```



```

# Path to the raw images directory
raw_data_dir = "intel_data/raw"
target_base_dir = "intel_data"

# Define the classes
classes = ["buildings", "forest", "glacier", "mountain", "sea", "street"]

# Define the split ratios
train_ratio = 0.7
val_ratio = 0.15
test_ratio = 0.15

# Create directories if they don't exist
for split in ["train", "val", "test"]:
    for cls in classes:
        os.makedirs(os.path.join(target_base_dir, split, cls), exist_ok=True)

# If the raw directory doesn't exist, create it and add placeholder images
if not os.path.exists(raw_data_dir):
    os.makedirs(raw_data_dir)
    for cls in classes:
        class_dir = os.path.join(raw_data_dir, cls)
        os.makedirs(class_dir, exist_ok=True)

        # Create a dummy text file for each "image"
        for i in range(10): # Assuming 10 dummy images per class
            with open(os.path.join(class_dir, f"image_{i}.jpg"), 'w') as f:
                f.write(f"Dummy content for {cls} image {i}")

print("Raw data directories created and placeholder files added.")

# Organize data for each class
for cls in classes:
    class_dir = os.path.join(raw_data_dir, cls)

    # Check if the class directory exists
    if not os.path.exists(class_dir):
        print(f"Directory not found: {class_dir}")
        continue # Skip to the next class if directory doesn't exist

    images = os.listdir(class_dir)

    # Split the images into train, validation, and test sets
    train_images, temp_images = train_test_split(images, test_size=(1 - train_ratio), random_state=42)
    val_images, test_images = train_test_split(temp_images, test_size=(test_ratio / (test_ratio + val_ratio)), random_state=42)

    # Copy images to respective directories
    for image_set, split in zip([train_images, val_images, test_images], ["train", "val", "test"]):
        for image_name in image_set:
            source_path = os.path.join(class_dir, image_name)
            target_path = os.path.join(target_base_dir, split, cls, image_name)
            shutil.copy2(source_path, target_path)

print("Data organized into train, val, and test directories.")

import torch
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# Define transformations for data augmentation (for training set)
train_transforms = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]), # Normalization for pre-trained models
])

# Define transformations for validation and test sets (no augmentation)
val_test_transforms = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])

# Load datasets using ImageFolder
train_data = datasets.ImageFolder(root='intel_data/train', transform=train_transforms)
val_data = datasets.ImageFolder(root='intel_data/val', transform=val_test_transforms)
test_data = datasets.ImageFolder(root='intel_data/test', transform=val_test_transforms)

```

```

# Create data loaders
train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
val_loader = DataLoader(val_data, batch_size=32, shuffle=False)
test_loader = DataLoader(test_data, batch_size=32, shuffle=False)

import torch.nn as nn
import torchvision.models as models

# Load a pre-trained ResNet model (e.g., ResNet18, ResNet34, etc.)
model = models.resnet18(pretrained=True)

# Freeze the weights of the initial layers
for param in model.parameters():
    param.requires_grad = False

# Replace the final fully connected layer to match the 6 classes in the dataset
model.fc = nn.Linear(model.fc.in_features, 6)

# Move the model to the appropriate device (GPU or CPU)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = model.to(device)

pip install ray[tune]

import ray
from ray import tune
from torch.optim import Adam, SGD
from torch.optim.lr_scheduler import StepLR
import torch.optim as optim
import torch.nn.functional as F

# Define the training function for Ray Tune
def train_fn(config):
    # Create the model, loss, and optimizer
    model = models.resnet18(pretrained=True)
    for param in model.parameters():
        param.requires_grad = False
    model.fc = nn.Linear(model.fc.in_features, 6)

    model = model.to(device)

    # Choose optimizer based on config
    optimizer_type = config["optimizer"]
    if optimizer_type == "adam":
        optimizer = Adam(model.fc.parameters(), lr=config["lr"])
    else:
        optimizer = SGD(model.fc.parameters(), lr=config["lr"], momentum=0.9)

    criterion = nn.CrossEntropyLoss()

    # Training loop
    for epoch in range(config["epochs"]):
        model.train()
        running_loss = 0.0
        correct = 0
        total = 0
        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)

            optimizer.zero_grad()

            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            running_loss += loss.item()
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

        # Validation
        model.eval()
        val_loss = 0.0
        correct = 0
        total = 0
        with torch.no_grad():

```

```

        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            val_loss += loss.item()
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    val_accuracy = correct / total
    tune.report(loss=running_loss / len(train_loader), accuracy=val_accuracy)

# Hyperparameter search space
search_space = {
    "lr": tune.grid_search([1e-3, 1e-4, 1e-5]),
    "epochs": 10,
    "batch_size": 32,
    "optimizer": tune.choice(["adam", "sgd"]),
}

# Start Ray Tune experiment
ray.init(ignore_reinit_error=True)

analysis = tune.run(
    train_fn,
    config=search_space,
    num_samples=1, # You can increase num_samples for more trials
    resources_per_trial={"cpu": 1, "gpu": 1},
)

# Print the best result
print("Best hyperparameters found were: ", analysis.best_config)

# Evaluate on the test set after hyperparameter tuning
best_model = models.resnet18(pretrained=True)
for param in best_model.parameters():
    param.requires_grad = False
best_model.fc = nn.Linear(best_model.fc.in_features, 6)
best_model = best_model.to(device)

optimizer = Adam(best_model.fc.parameters(), lr=analysis.best_config["lr"])
criterion = nn.CrossEntropyLoss()

# Load best configuration
best_model.eval()
test_loss = 0.0
correct = 0
total = 0
with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = best_model(images)
        loss = criterion(outputs, labels)
        test_loss += loss.item()
        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

test_accuracy = correct / total
print(f"Test Accuracy: {test_accuracy:.4f}")

```